

FUNDAMENTOS DE PROGRAMAÇÃO

Array



Teach
3035

Índice

Array.....	03
Criando arrays.....	04
Acessando, alterando e adicionando elementos ao array utilizando o índice.....	04
Propriedades e métodos de arrays.....	05
Adicionar elementos.....	05
Remover elementos.....	06

Array



Os **arrays** são usados para armazenar múltiplos elementos em uma única variável. Na prática, um array é uma estrutura de dados que associa cada elemento a um índice numérico. Esses elementos podem ser de qualquer tipo primitivo de dados, objetos ou até mesmo outros arrays.

Criando arrays



Existem várias maneiras de criar um array em JavaScript. A forma mais comum é usando a sintaxe de colchetes [] e separando os elementos por vírgulas, como o exemplo abaixo:

```
let cars = [ "Fiat", "Chevrolet", "Volkswagen" ];
```

O outro exemplo abaixo também cria um array e atribui valores a ele:

```
let cars = new Array("Fiat", "Chevrolet", "Volkswagen");
```

Por simplicidade, legibilidade e velocidade de execução, use o primeiro!

Acessando, alterando e adicionando elementos ao array utilizando o índice



Para acessar os elementos de um array em JavaScript, é utilizado um índice baseado em zero. Isso significa que o primeiro elemento possui o índice 0, o segundo elemento tem o índice 1 e assim por diante. Portanto, para acessar um elemento de um array, siga o exemplo abaixo:

```
console.log(cars[0]); // Saída: Fiat  
console.log(cars[2]); // Saída: Volkswagen
```

Você também pode alterar o valor de um elemento acessando-o pelo índice:

```
cars[0] = "Hyundai"  
console.log(cars) // Saída: [Hyundai, Chevrolet, Volkswagen]
```

Da mesma forma, você pode adicionar elementos em posição que ainda não existe, veja:

```
cars[3] = "Toyota"  
console.log(cars) // Saída: [Hyundai, Chevrolet, Volkswagen, Toyota]
```

Propriedades e métodos de arrays



Os arrays em JavaScript possuem diversas propriedades e métodos que facilitam a manipulação e o acesso aos elementos armazenados. Aqui estão alguns exemplos dessas propriedades e métodos:

length: retorna o número de elementos do array

```
console.log(cars.length) // saída: 4
```

Adicionar elementos



push(): adiciona elementos no final do array

```
cars.push('Renault');  
console.log(cars) // saída: [Hyundai, Chevrolet, Volkswagen, Toyota,  
Renault]
```

unshift(): adiciona elementos no início do array

```
cars.push('Ford');  
console.log(cars) // saída: [Ford, Hyundai, Chevrolet, Volkswagen, Toyota,  
Renault]
```

Remover elementos



pop(): remove o último elemento do array e retorna o valor removido

```
cars.pop();  
console.log(cars) // saída: [Chevrolet, Volkswagen, Toyota, Renault]
```

shift(): remove o primeiro elemento do array

```
cars.shift();  
console.log(cars) // saída: [Volkswagen, Toyota, Renault]
```

slice(): retorna uma cópia superficial (shallow copy) de parte do array, especificada pelos índices de início e fim (fim não incluso).

```
let cars = [ "Fiat", "Chevrolet", "Volkswagen" ];  
let filteredCars = cars.slice(0,2);  
console.log(cars) // saída: [Fiat, Chevrolet]
```

Veja uma lista completa de todos os métodos clicando [aqui](#)

Remover elementos



▼ Métodos

`Array.prototype[@@iterator]()`

`Array.prototype.at()`

`Array.prototype.concat()`

`Array.prototype.copyWithin()`

`Array.prototype.entries()`

`Array.prototype.every()`

`Array.prototype.fill()`

`Array.prototype.filter()`

`Array.prototype.find()`

`Array.prototype.findIndex()`

`Array.prototype.findLast()` (en-US)

`Array.prototype.findLastIndex()` (en-US)

`Array.prototype.flat()`

`Array.prototype.flatMap()`

`Array.prototype.forEach()`

`Array.from()`

`Array.fromAsync()` (en-US)

`Array.prototype.group()` (en-US) 

`Array.prototype.groupToMap()` (en-US)



`Array.prototype.includes()` :